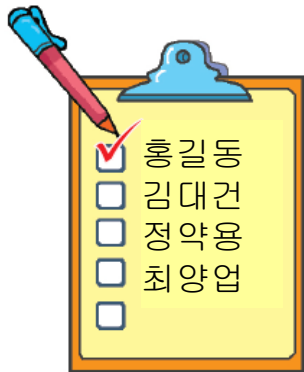


CHAP 7: 리스트

리스트 생성, 리스트의 인덱싱,
리스트의 슬라이싱,
리스트 복사,
리스트 연산(+ 연결연산, * 반복연산, in, not in)
리스트의 메소드, 리스트 삭제

리스트 생성

- 여러 개의 데이터들을 하나로 묶어서 저장하고 처리할 수 있다.
- 하나의 이름을 부여,
인덱스 번호를 사용하여 각각의 데이터에 접근.



```
members= [ '홍길동','김대건','정약용','최양업' ]
```

(출처: 두근두근 파이썬)

리스트 생성

- 학교 건물 이름을 저장하여 관리한다고 하자
 - 각자 서로 다른 변수명을 사용해서 저장할 수 있다.

예)

```
>>> building1 = '로올라도서관'  
>>> building2 = '김대건관'  
>>> building3 = '마태오관'
```

- 리스트를 이용하면 여러 데이터를 함께 저장하여, 변수명을 여러 개 사용할 필요가 없다.

예)

```
>>> buildings = [ '로올라도서관', '김대건관', '마태오관' ]
```



(출처: 두근두근 파이썬)

리스트 생성

- 여러 개의 데이터들을 하나로 묶어서 저장하고 처리할 수 있다.
 - 원소에 차례가 있음.(sequence object)
 - 인덱싱(indexing)과 슬라이싱(slicing) 사용.
- 형식

```
변수 = [요소1, 요소2, 요소3, ...]
```

- 대괄호 []안에 콤마로 구분하여 원소들을 표시.
 - 리스트의 원소는 수정 가능하다.
 - 원소 값으로 어떠한 자료형 값도 올 수 있다.
-
- 빈 리스트 생성
letters= []
letters= list()

```
>>> a2 = [] # 빈 리스트 생성 후, 원소를 추가.  
>>> a2.append('red') # append()메소드로 원소 추가  
>>> a2.append('green') # append()메소드로 원소 추가  
>>> a2.append('blue')  
>>> a2  
['red', 'green', 'blue']
```

리스트 생성

- 예) 영문 알파벳 대문자 모음을 저장.

```
vowels = [ 'A', 'E', 'I', 'O', 'U' ]
```

- 예) 최근 4일동안 달러환율 저장. 1190, 1191, 1198, 1195원을 저장.

```
price = [ 1190, 1191, 1198, 1195 ]
```

- 예) 이름과 전화번호 저장

```
myPhone = [ '홍길동', '010-1234-5678' ]
```

#다른 자료형을 원소로 사용 가능

리스트 생성

- 리스트 생성하기
 - [] 안에 항목(원소)들을 적어준다.
 - list() 함수에 인수를 지정하여 생성.
 - 인수로는 원소를 갖는 **iterable** 데이터 1개만 올 수 있다.
 - 인수로 지정된 값의 원소들을 리스트의 원소로 구성.
 - 형식

```
변수 = list( iterable자료형 )
```



반복가능객체

원소를 가지고 있는 객체.
문자열, 튜플, 집합, 사전,
range()함수의 반환값 등등.

리스트 생성

- 예) 리스트 생성하기

```
>>> a1 = ['red', 'green', 'blue'] # 리스트 생성
>>> a1
['red', 'green', 'blue']
>>> print(a1)
['red', 'green', 'blue']
>>> a2 = [] # 빈 리스트 생성 후, 원소를 추가.
>>> a2.append('red') # append()메소드로 원소 추가
>>> a2.append('green') # append()메소드로 원소 추가
>>> a2 = a2 + ['blue'] # +연산자로 원소 추가
>>> a2
['red', 'green', 'blue']
>>>
```

리스트 생성

- 예) 리스트 생성

```
>>> a3 = list( 'AEIOU' ) #리스트 생성. ['A','E','I','O','U']
```

```
>>> a3 #문자열의 원소 → 리스트의 원소
```

```
['A','E','I','O','U']
```

```
>>>
```

```
>>> a3 = ['A','E','I','O','U'] # 리스트 생성
```

```
>>> print(a3)
```

```
['A','E','I','O','U']
```

```
>>>
```

```
>>> items = "이삭 야곱 요셉".split() # split()함수 값이 리스트.  
>>> items # 분리된 값들이 원소로 구성.
```

```
['이삭', '야곱', '요셉']
```

```
>>> dates = '12/25/2018'.split('/') # split()함수 값이 리스트.
```

```
>>> dates
```

```
['12', '25', '2018']
```


리스트 생성

```
>>> a = "Obey to the truth."
>>> a.split()    #4개로 분리 됨
['Obey', 'to', 'the', 'truth.']
>>>
>>> w1, w2, w3, w4= a.split() # 4개의 변수에 각각 할당
>>>                        # w1='Obey' , w2='to' , w3='the' , w4='truth.'
>>>
>>> w= a. split()  #w= ['Obey', 'to', 'the', 'truth.']
>>> w
['Obey', 'to', 'the', 'truth.']
```

리스트 생성

- 예) `list()` 함수의 잘못된 사용. 리스트 객체 `[2,3,4]` 생성하기.

```
>>> list1 = list( 2,3,4 )
```

Traceback (most recent call last):

File "<pyshell#3>", line 1, in <module>

```
list1=list(2,3,4)
```

TypeError: list() takes at most 1 argument (3 given)

```
>>>
```



- `list()` 함수의 올바른 사용

```
list1 = list( {2,3,4} ) #인수가 set
```

또는

```
list1 = list( (2,3,4) ) #인수가 tuple
```

또는

```
list1 = list( [2,3,4] ) #인수가 list
```

리스트 생성

- 예) 올림픽경기의 나라별 메달 개수

```
나라 = [ 'Austria', 'Canada', 'China', 'France', 'Germany',  
         'Japan', 'Korea', 'Norway', 'Russia', 'Viet Nam' ]
```

```
금 = [ 0, 4, 5, 0, 1, 1, 5, 4, 4, 1]
```

```
은 = [ 2, 8, 0, 4, 1, 1, 3, 4, 6, 1]
```

```
동 = [ 1, 5, 1, 5, 2, 0, 1, 7, 4, 9]
```

```
medals= [ ['Austria',0,2,1], ['Canada',4,8,5], ['China',5,0,1],  
          ['France',0,4,5], [ 'Germany',1,1,2], ['Japan',1,1,0],  
          ['Korea',5,3,1], ['Norway',4,4,7], ['Russia',4,6,4],  
          ['Viet Nam',1,1,9] ]
```

리스트의 연산: in, not in

- in, not in 연산자 - 리스트에 어떤 값의 포함 여부를 알려줌
 - item in list객체 item이 list에 있으면 True, 없으면 False.
 - item not in list객체 item이 list에 없으면 True, 있으면 False.
- 예) 명단에 포함 여부를 알려주기

test.py

```
passName= ['연주', '요셉', '길동',  
           '철수', '영희', '소연']
```

```
check= input('Enter: ')
```

```
if check in passName :  
    print("합격!")  
else :  
    print("불합격")
```

😊 실행 화면

Enter: 요셉 
합격!

Enter: 길 
불합격

Enter: 영주 
불합격

리스트의 연산

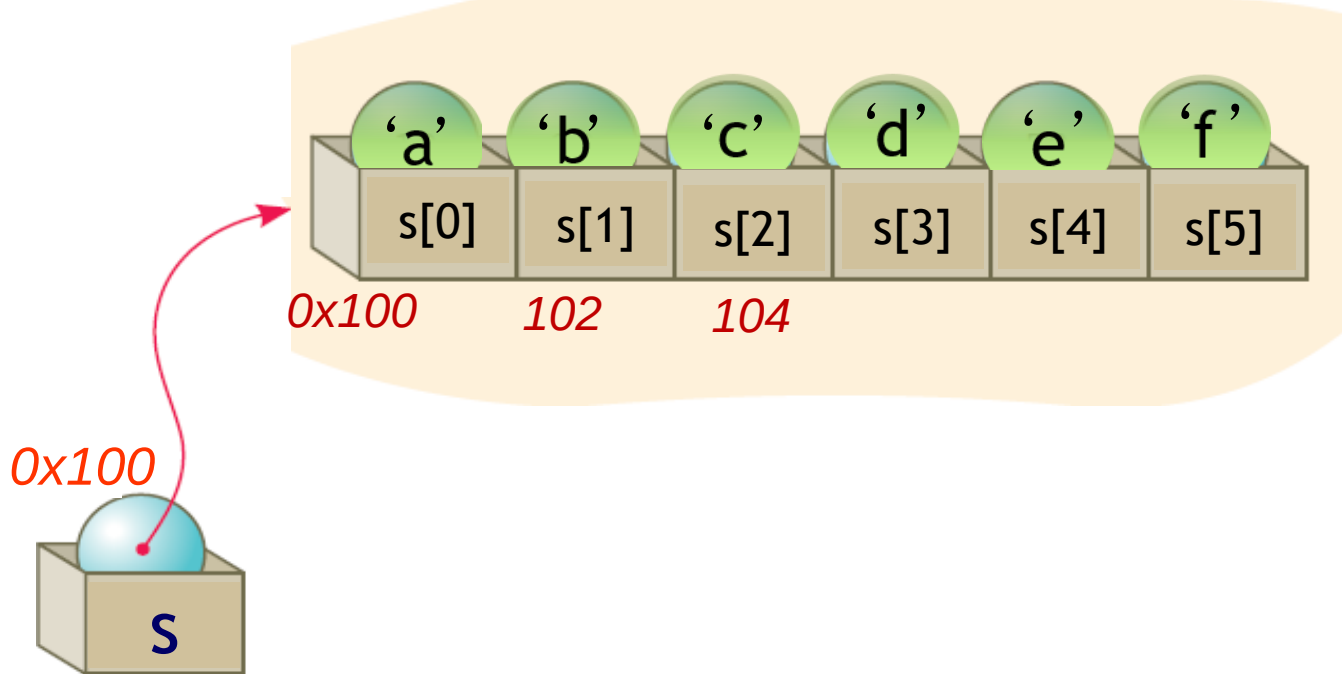
- + 연산자 **list객체 + list객체**
 - 리스트들을 연결하여 새로운 리스트로 만든다.
 - 원소끼리 더해지는 것이 아님.
- * 연산자 **list객체 * n**
 - 리스트의 원소를 n번 반복하여 새로운 **list**를 만든다.
 - 원소 값에 곱셈을 하는 것이 아님

```
>>> L = [1, 3, 5] ; M = [2, 4, 6]
>>> L + M    # 리스트 L과 M을 합쳐서 새 리스트 생성
[1, 3, 5, 2, 4, 6]
>>> p = L + M    # [1, 3, 5, 2, 4, 6]
>>>
>>> L * 3    # 리스트 L을 3회 반복하여 새 리스트 생성
[1, 3, 5, 1, 3, 5, 1, 3, 5]
>>> q = L * 3
>>> print(q)
[1, 3, 5, 1, 3, 5, 1, 3, 5]
```

리스트의 인덱싱(Indexing)

● `s = [ 'a', 'b', 'c', 'd', 'e', 'f' ]`

- 리스트의 인덱스는 0부터.
- `s[0] ~ s[5]`
- 즉, 리스트 내의 원소는
리스트의 첫 원소를 상대적 위치로 해서 참조.

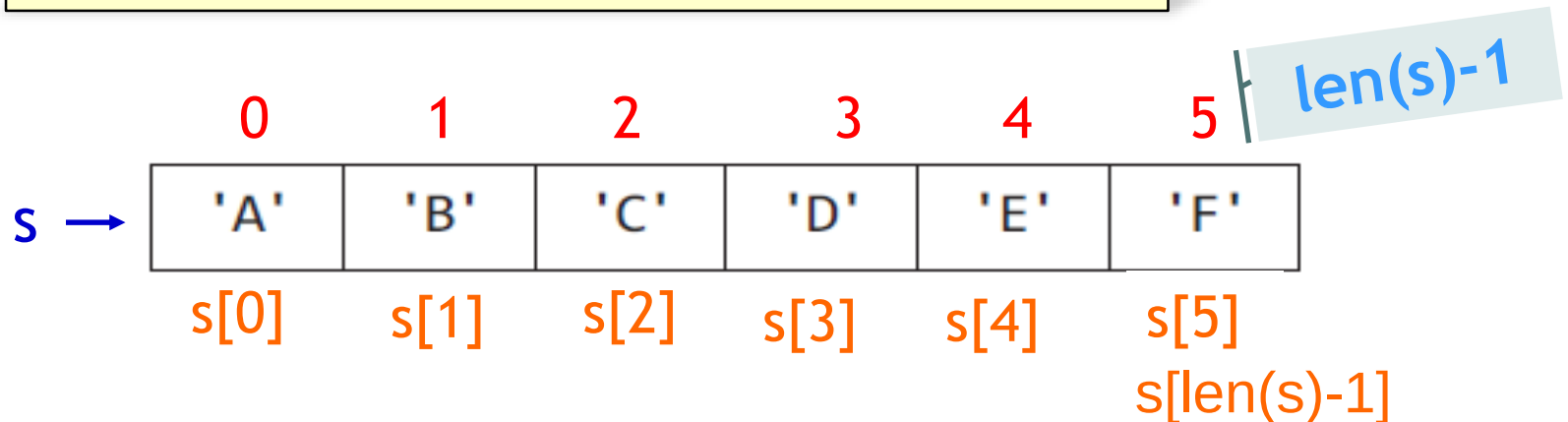


리스트의 참조 값(주소)를
갖고 있음.

리스트 indexing

- 인덱스 값을 통해 특정 원소에 접근할 수 있다.
- 인덱스 범위
 - 0 ~ $\text{len}(\text{리스트 변수})-1$
 - $\text{len}(\text{리스트})$: 크기 즉, 원소의 개수를 알려주는 함수.
- 대괄호 [] 안에 Index를 지정하여, 원소 추출 및 수정할 수 있다.
 - $[0] \sim [\text{len}(\text{리스트 변수})-1]$

예) `>>> s = ['A', 'B', 'C', 'D', 'E', 'F']`

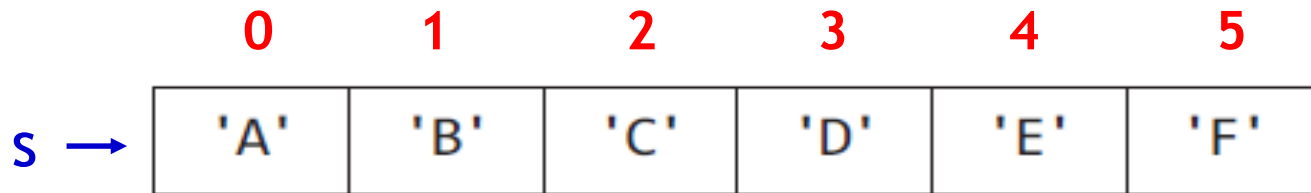


리스트 indexing

예) `>>> s = ['A', 'B', 'C', 'D', 'E', 'F']`

```
>>> len(s) #size of List. 6
6
```

- index는 `[0] ~ [len(s)-1]`. 즉, `[0] ~ [5]`
- 마지막 원소의 인덱스번호: `len(s)-1`



`>>> s[1]` Enter
'B'
>>>

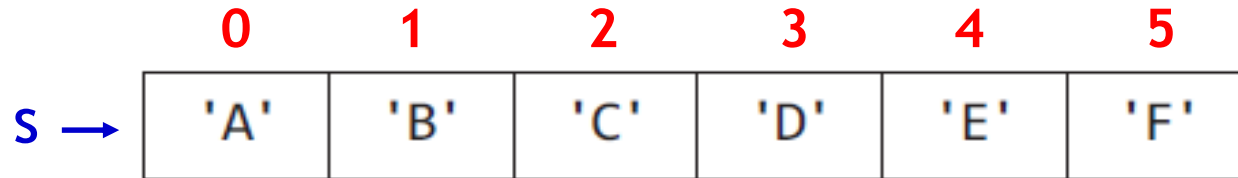
`>>> s[5]` Enter
'F'
>>>

`s[len(s)-1]`

리스트 indexing

- 예) 인덱싱으로 원소 수정하기

```
s = ['A', 'B', 'C', 'D', 'E', 'F']
```

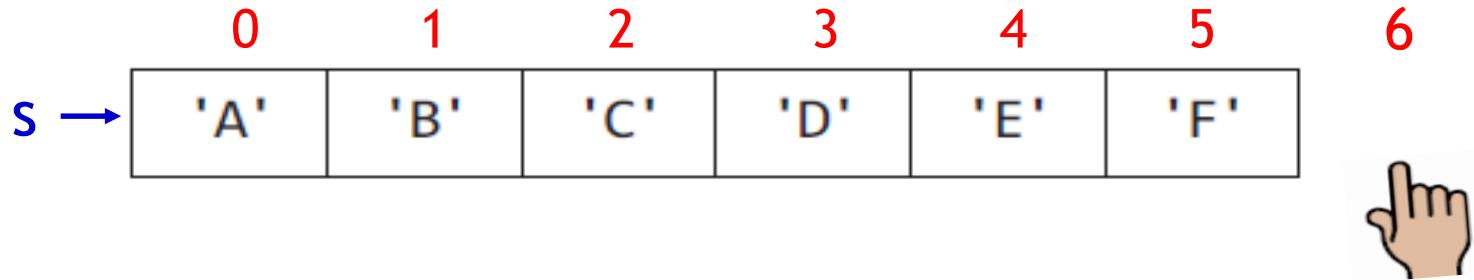


```
>>> s = ['A', 'B', 'C', 'D', 'E', 'F'] 
>>> s[2]  #원소 참조
'C'
>>> s[2] = 'Christmas'  #원소 수정
>>> s 
['A', 'B', 'Christmas', 'D', 'E', 'F']
>>>
```

리스트 indexing

- 정수가 아닌 index를 사용하면 **TypeError**, 범위를 벗어나면 **IndexError**가 발생한다.

```
s = ['A', 'B', 'C', 'D', 'E', 'F']
```

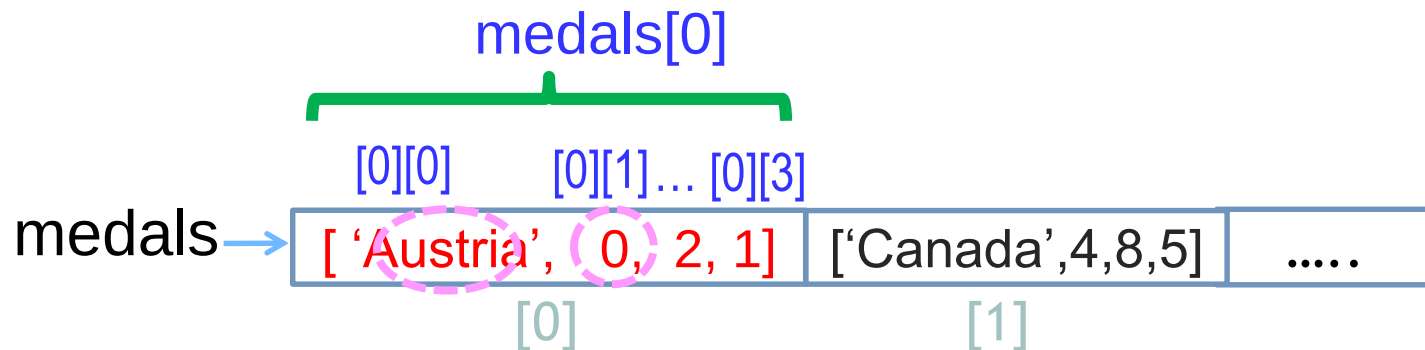


```
>>> s[6]      # IndexError (out of range)
Traceback (most recent call last):
  File "<pyshell#12>", line 1, in <module>
    s[6]IndexError: list index out of range
>>>
```

리스트 indexing

- (예) 복잡한 리스트 indexing

```
medals= [ ['Austria',0,2,1], ['Canada',4,8,5], ['China',5,0,1],  
          ['France',0,4,5], ['Germany',1,1,2], ['Japan',1,1,0],  
          ['Korea',5,3,1], ['Norway',4,4,7], ['Russia',4,6,4],  
          ['Viet Nam',1,1,9] ]
```



```
>>> medals[0]  
[ 'Austria', 0, 2, 1 ]  
>>> medals[0][0]  
'Austria'  
>>> medals[0][1]  
0  
>>>
```

리스트 indexing

- 인덱싱으로 리스트의 원소를 수정할 수 있다.
- 예) 캐나다의 금메달 수를 6개로 수정하세요.
해당 원소 금[1]을 6으로 수정??

금 →

0	1	2	3	4	5	6	7	8	9
0	4	5	0	1	1	5	4	4	1

```
>>> 금 = [0, 4, 5, 0, 1, 1, 5, 4, 4, 1]
>>>
>>> 금[1] = 6 # 원소 수정. 금[1]을 6으로 수정
>>>
>>> 금[1]
6
>>> 금
[0, 6, 5, 0, 1, 1, 5, 4, 4, 1]
>>>
```

리스트 indexing

- 예) 캐나다가 금메달 **2개를 추가**했다고 가정하자.
해당 원소를 수정??

```
gold = [ 0, 4, 5, 0, 1, 1, 5, 4, 4, 1]
```

```
# 원소를 수정하기
```

```
gold[1] = gold[1]+2
```

```
print(gold)      # [0, 6, 5, 0, 1, 1, 5, 4, 4, 1]
```

출력 화면

```
[0, 6, 5, 0, 1, 1, 5, 4, 4, 1]
```

gold →

0	4	5	0	1	1	5	4	4	1
---	---	---	---	---	---	---	---	---	---

6



원소 수정 후

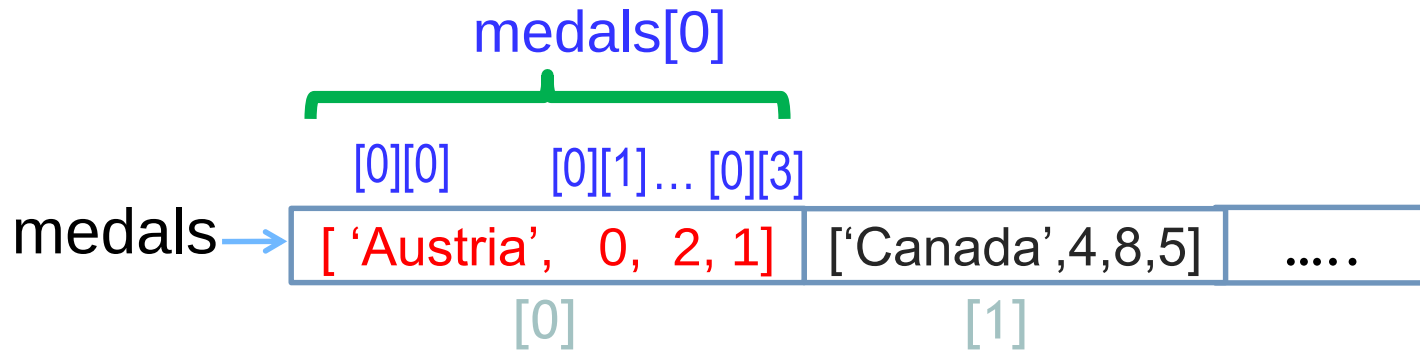
gold →

0	6	5	0	1	1	5	4	4	1
---	---	---	---	---	---	---	---	---	---

리스트 indexing

- 예) 인덱싱으로 리스트의 원소 수정하기. (복잡한 리스트 indexing)

```
medals= [ ['Austria',0,2,1], ['Canada',4,8,5], ['China',5,0,1],  
          ['France',0,4,5], ['Germany',1,1,2], ['Japan',1,1,0],  
          ['Korea',5,3,1], ['Norway',4,4,7], ['Russia',4,6,4],  
          ['Viet Nam',1,1,9] ]
```



```
>>> medals[0][1] += 2  
>>> medals[0][2] += 3  
>>> print( medals[0] )  
['Austria',2,5,1]  
>>>
```

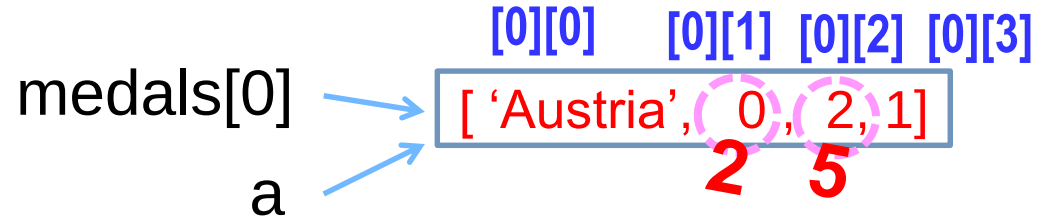


리스트 indexing

- 리스트에서 원소를 수정

```
medals= [ ['Austria',0,2,1], ['Canada',4,8,5], ['China',5,0,1],  
          ['France',0,4,5], ['Germany',1,1,2], ['Japan',1,1,0],  
          ['Korea',5,3,1], ['Norway',4,4,7], ['Russia',4,6,4],  
          ['Viet Nam',1,1,9] ]
```

```
>>> medals[0]  
[ 'Austria', 0, 2, 1 ]  
>>> a = medals[0]  
>>> a  
[ 'Austria', 0, 2, 1 ]  
>>> a[1] = a[1]+2  
>>> a[2] = a[2]+ 3  
>>> a  
[ 'Austria',2,5,1]  
>>> medals[0]  
[ 'Austria',2,5,1]  
>>>
```



```
>>> medals[0][1] = medals[0][1] + 2  
>>> medals[0][2] = medals[0][2] + 3  
>>> print( medals[0] )  
[ 'Austria',2,5,1]  
>>>
```

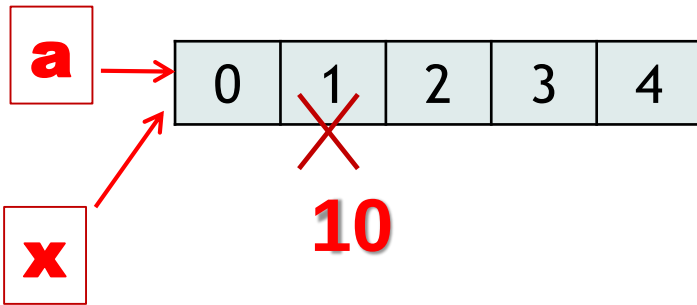
리스트 indexing

- 리스트에서 원소를 수정
 - 변수 **a**를 통해 수정된 값이 다른 이름(변수) **x**에서도 보임

```
>>> a = [0, 1, 2, 3, 4]
```

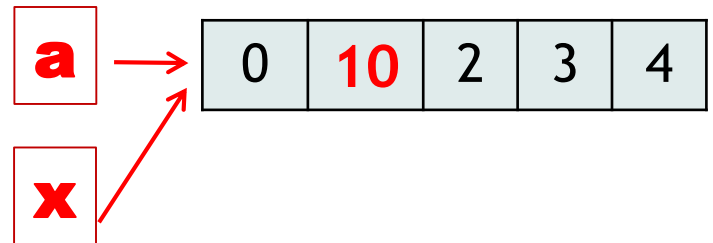
```
>>> x = a
```

```
>>> a[1] = 10 #원소 수정
```



원소 수정 후

변수 **a**를 통해 수정된 값이
다른 이름 **x**에서도 보임



```
>>> a[1]
```

```
10
```

```
>>> x[1]
```

```
10
```

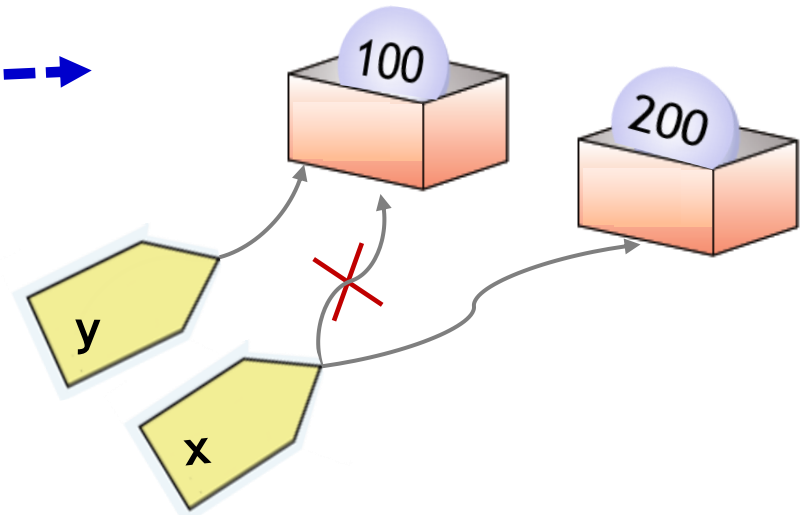
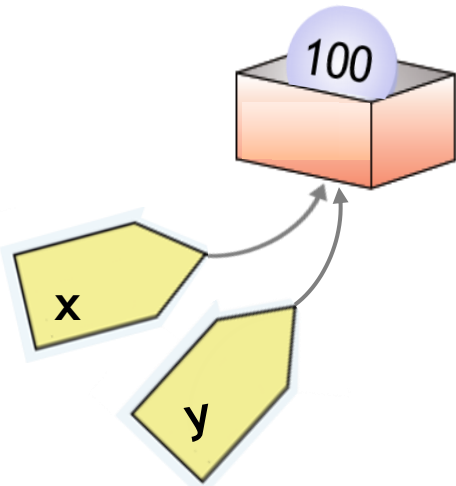
```
>>> print(x)
```

```
[0,10,2,3,4]
```


[비교] 정수형 변수

```
>>> x = 100
>>> y = x
>>> x = 200
>>> print(x)
200
```

```
>>> x
200
>>> y
100
>>>
```

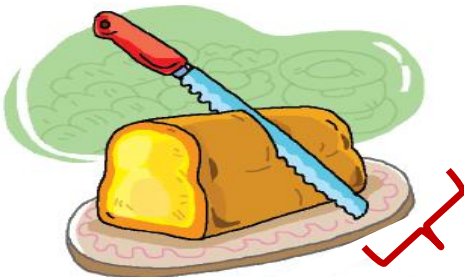


변수 x를 통해 수정된 값이
다른 이름 y에서 보이지 않음.

리스트 Slicing

- List의 일부 원소들을 추출해서 새로운 리스트를 만든다.
 - slicing 결과는 리스트이다. (원소가 없는 빈 리스트일 수 있음)
 - slicing을 통하여 일부 원소를 참조만 한다.
원본 리스트는 변하지 않는다.
- Syntax (L을 n개의 item을 갖는 list라고 가정.)

`L[b : e : s]` # `b = begin`, `e = end`, `s = step`을 의미.



(출처: 두근두근 파이썬)

(예)

```
>>> letters = [ 'A', 'B', 'C', 'D', 'E' ]  
>>> letters[0:3]  
['A', 'B', 'C']
```

letters[0:3]

리스트 Slicing

$L[b : e : s]$ # $b = \text{begin}$, $e = \text{end}$, $s = \text{step}$ 을 의미.

- Slicing Rule ($b, e \geq 0$ 인 경우로 설명)

- $s > 0$ ($b < e$ 이어야 한다. 아니면 빈 리스트 생성)

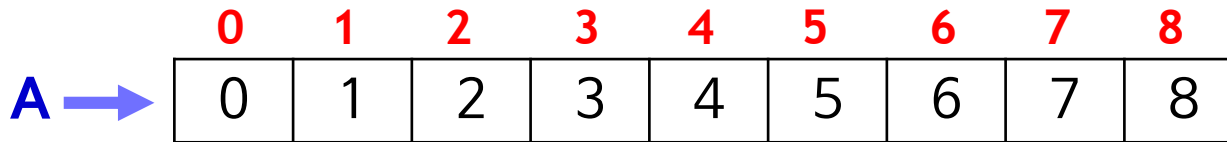
- Index를 b 부터 $e-1$ 까지 s 씩 증가하며 리스트를 참조한다.
- b 를 생략하면 0부터, e 를 생략하면 끝까지, s 를 생략하면 +1씩.
 $L[: e : s]$ $L[b : : s]$ $L[b:e]$

- $s < 0$ ($b > e$ 이어야 한다. 아니면 빈 리스트 생성)

- Index를 b 부터 $e+1$ 까지 $|s|$ 씩 감소하며 리스트를 참조한다.
- b 를 생략하면 끝에서부터, e 를 생략하면 첫 번째 원소까지.
 $L[: e : s]$ $L[b : : s]$

리스트 Slicing

- 예) $A = [0, 1, 2, 3, 4, 5, 6, 7, 8]$ # 원소 값이 정수.



- step이 1 인 경우 (default값),

$A[:]$ # $[0,1,2,3,4,5,6,7,8]$ begin=0, end=9, step=1

$A[:5]$ # $[0,1,2,3,4]$ begin=0, end=5, step=1
 $A[0:5]$ # $[0,1,2,3,4]$
 $A[0:5:1]$ # $[0,1,2,3,4]$

} 동일

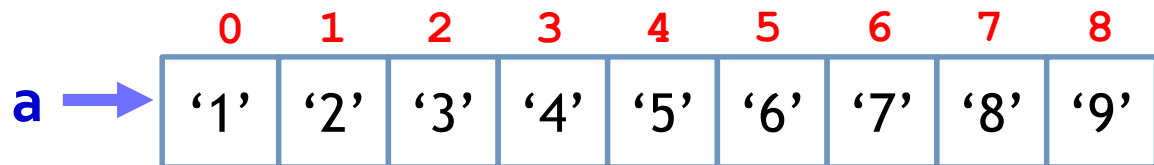
$A[6:8]$ # $[6,7]$ begin= 6, end= 8, step= 1

$A[4:5]$ # $[4]$ a[4]을 선택

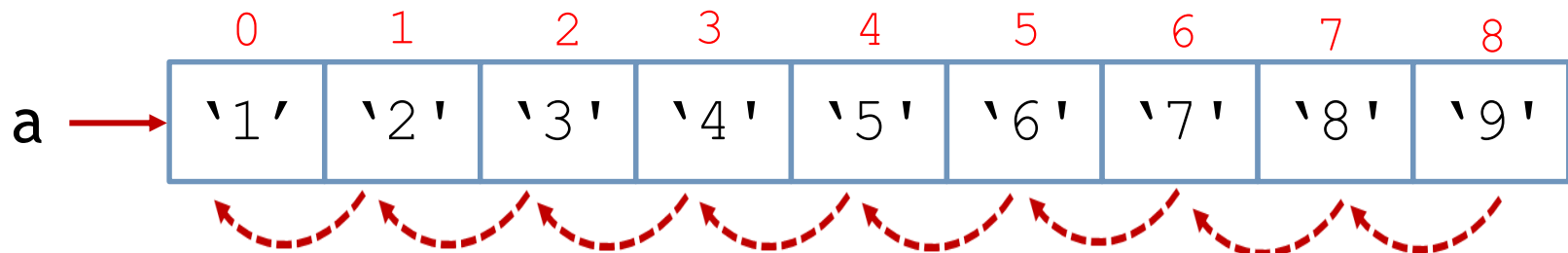
리스트 Slicing

- 예) `a = ['1', '2', '3', '4', '5', '6', '7', '8', '9']`

- $s < 0$ 인 경우



```
>>> a = ['1','2','3','4','5','6','7','8','9']
>>> a[::-1] # 전체를 거꾸로 추출.
['9', '8', '7', '6', '5', '4', '3', '2', '1']
>>>
```

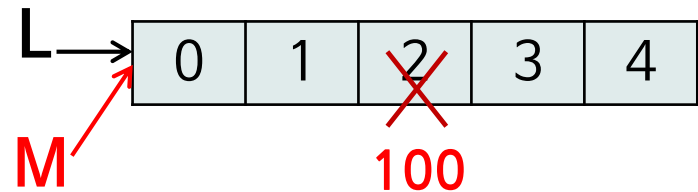


step이 -1이고, -1 -1
마지막원소에서 첫번째 원소까지 추출

리스트의 복사

- 리스트 참조를 복사하기
 - 변수에 리스트 변수를 할당하면, 리스트 객체의 참조가 복사됨. 즉, 동일한 객체를 다른 이름으로 참조하게 된다.
- 예) 동일한 객체를 다른 이름으로 참조

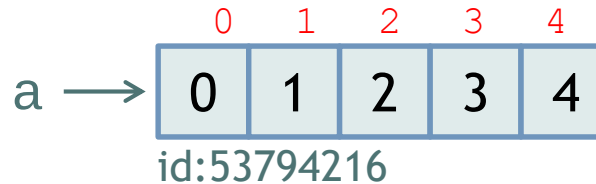
```
>>> L = [0,1, 2, 3, 4]
>>> M = L   #같은 객체를 참조
>>> M[2] = 100 #원소 수정
>>> print(L)
[0, 1, 100, 3, 4]
>>> print(M)
[0, 1, 100, 3, 4]
```



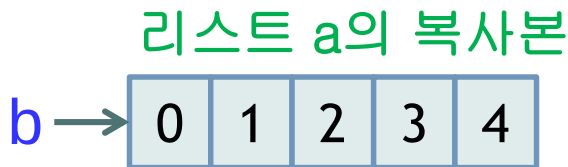
리스트의 복사

- 리스트 복사본 만들기
 - 슬라이싱 사용: 리스트 전체를 추출해서, 복사본을 만든다
 - `copy()` 메소드: 리스트 객체를 복사

- 예) 리스트의 복사본 만들기

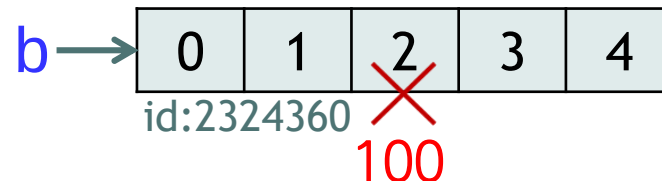


```
>>> a = [0,1,2,3,4]
>>> b = a[:] #전체 추출
>>> b
[0,1,2,3,4]
```



```
>>> a = [0,1,2,3,4]
>>> b = a.copy() #b=a[:]
```

```
>>> b[2] = 100 #원소수정
>>> a
[0, 1, 2, 3, 4]
>>> b
[0, 1, 100, 3, 4]
```



리스트의 Method

- 리스트에서 특정 항목의 **인덱스 번호(위치)** 찾기
 - `index(x)` - 리스트에서 데이터 `x`의 **인덱스(위치)**를 반환한다.
- 데이터 개수 세기
 - `count(x)` - 리스트에서 데이터 `x`의 **개수**를 반환한다.

```
>>> L = ['M','a','r','a','n','a','t','h','a']
>>> L.index('r')
2
>>> L.index('a') #여러 개이면 첫 번째 인덱스 반환
1
>>> L.count('g')
0
>>> L.count('a')
4
```


리스트의 Method

- 리스트에 데이터 추가하기
 - `append( x )` - 데이터 `x`를 리스트 끝에 추가한다.
 - `insert( i,x )` - 주어진 위치 인덱스 i에 데이터 x를 삽입한다.

```
>>> L = ['a', 'b', 'c', 'd']
```

```
>>> L.append('e') # 'e'를 끝에 추가.
```

```
>>> print(L)
```

```
['a', 'b', 'c', 'd', 'e']
```

```
>>> L.insert(2,'bee') # 'aa'을 인덱스2에 삽입.
```

```
>>> print(L)
```

```
['a', 'b', 'bee', 'c', 'd', 'e']
```

리스트의 Method

- 리스트에서 데이터 삭제하기
 - `pop()` - 리스트의 맨 마지막 데이터를 반환하고 삭제한다.
 - `pop(i)` - 리스트에서 인덱스 i의 원소를 반환하고 삭제한다.
 - `remove(x)` - 리스트에서 데이터 x를 삭제한다.
x가 여러 개 있으면 맨 처음 x를 삭제한다.

```
>>> L = ['a', 'b', 'c', 'd', 'e']
>>> L.pop() # 마지막 원소를 제거 및 값 반환
'e'
>>> print(L)
['a', 'b', 'c', 'd']
>>> L.pop(1) # index 1번 원소를 제거 및 값 반환
'b'
>>> print(L)
['a', 'c', 'd']
>>> L.remove('c') # 값이 'c'인 원소 제거. ['a', 'd']
>>>
```

리스트의 Method

- 리스트 정렬하기 - `sort()`

```
>>> L = [4,5,1,2,3]
```

```
>>> L.sort() # 오름차순 정렬
```

```
>>> print(L)
```

```
[1, 2, 3, 4, 5]
```

```
>>>
```

```
>>> L = [4,5,1,2,3]
```

```
>>> L.sort(reverse=True) # 내림차순 정렬.
```

```
>>> print(L)
```

```
[5, 4, 3, 2, 1]
```



(출처: 두근두근 파이썬)

리스트 삭제

- del

- 리스트의 원소, 리스트 일부, 또는 리스트 자체를 삭제
- 예) 원소 삭제

```
>>> a = [0, 1, 2, 3, 4, 5]
>>> del a[1] # a[1] 삭제. del a[1:2] 와 동일
>>> print(a)
[0, 2, 3, 4, 5]
```

- 예) 리스트의 일부를 삭제

```
>>> a = [0, 1, 2, 3, 4, 5]
>>> del a[3:] # a[3] 이후 모든 원소가 선택, 삭제됨
>>> print(a)
[0, 1, 2]
>>> del a[0:3] # 슬라이싱을 통해 a[0]~a[2] 선택, 삭제됨. []
>>> print(a)
[]
```

[참고 자료] 리스트의 Method

method	Description (x: 리스트, a: 임의의 object)
x.append(a)	데이터 a를 리스트 x의 끝에 추가
x.extend(L)	리스트 L의 모든 원소를 리스트 x의 마지막에 추가
x.insert(i, a)	a를 리스트 x의 index i에 추가
x.remove(a)	리스트 x에서 원소 값이 데이터 a인 첫 원소 제거. (반환 값 없음)
x.pop()	x의 마지막 원소 제거 및 반환
x.pop(i)	x[i]를 x에서 제거하고 그 값을 반환.
x.clear()	리스트 x의 모든 원소를 삭제. 빈 리스트가 됨
x.index(a)	리스트 x에서 원소 값이 a인 첫 번째 원소의 index를 반환
x.count(a)	리스트 x에서 a 값과 같은 원소의 개수를 반환
x.sort()	x의 원소들을 오름차순으로 정렬. 내림차순으로 정렬하려면 x.sort(reverse=True) 사용.
x.reverse()	x의 원소들을 역으로 재 배치 (정렬과 다름).
x.copy()	a shallow copy of the list (y = x[:]와 동일)

[참고자료] 내장함수

function	Description(인수 x : 리스트)
<code>enumerate(x)</code>	x의 모든 원소에 대해 튜플 (index, 원소 값)을 얻을 수 있는 enumerate object를 반환. index = 0, 1, 2, ...
<code>len(x)</code>	x의 원소 개수를 반환
<code>list(y)</code>	Iterable한 y를 리스트로 변환하여 반환
<code>max(x)</code>	x의 원소 중 최대 값 반환(비교 불가능하면 TypeError)
<code>min(x)</code>	x의 원소 중 최소 값 반환(비교 불가능하면 TypeError)
<code>sorted(x)</code>	x의 원소를 정렬한 리스트 반환. x는 불변이고 오름차순 또는 내림차순으로 정렬(<code>sorted(x, reverse=True)</code>)
<code>sum(x)</code>	x의 원소 합을 반환(더할 수 없으면 TypeError)